

JC1503: Object-Oriented Programming

Group Project: Game Inventory and Trading System

Group [MINGJIN LI XINGZHOU PENG JIAFENG YE
 LVZHEN ZHOU WEIJIE ZHOU YUXI ZHU]

May 8, 2026

Individual Contributions

Member Name	Responsibilities	Technical Implementation
MINGJIN LI	Market and Trading	Execution and filtering of pending orders, Stack, Structure-BST, Structure-Tree and Transaction implementation
XINGZHOU PENG	Inventory	Structure-Doublylinkedlist, Sorting Algorithm and Validation
JIAFENG YE	Item Management	Item-Implementation(crud), Polymorphism, Structure-HashMap and Structure-Tree
LVZHEN ZHOU	Overall Management, System Startup, Main Menu, History and Reports, Persistence, Test	File and Code Review, Data loading and Initialization, Entry and Exit, UI, Analytics, JSON Serialization, Save, Backups, Reset, Exception Management, Journal and so on
WEIJIE ZHOU	Player Management	Player System with CRUD, Read and Debug, Queue, Structure-HashMap
YUXI ZHU	Exceptions and Logs	Define the TradingSystemError Base Class, Standard Translation, Debug

Table 1: Summary of individual member responsibilities.

1 Problem and System Design

1.1 Application Domain

- **Scenario Description**

This project is a **Minecraft-themed inventory and player trading system** that simulates how items are stored, organized, and exchanged between players in the open-world sandbox game Minecraft.

- **Real-World Problem It Manages**

This system addresses the real-world problem of efficient personal asset tracking and secure peer-to-peer exchange of physical/digital goods. In real life, it directly translates:

- Personal inventory and asset management
- Secure peer-to-peer barter and trading systems
- Resource and supply tracking

- **Summary**

In short, the Minecraft inventory and trading system is a simplified but realistic model that tracks personal possessions and facilitates fair, structured direct trade — a common logistical and economic problem in both digital and physical environments.

1.2 System Architecture

- **Overall System Structure**

This project follows a layered, object-oriented architecture defined up-front in `docs/data-design.md` and `docs/services-interface.md`. The system is divided into six cooperating components, each mapped to a single top-level package so responsibilities never blur:

(1) Domain Model Layer (`src/models/`)

This layer defines all core entities and their relationships using OOP principles:

- Abstract base class `Item` with three behavioural mixins (`Durable`, `Equippable`, `Stackable`) and five polymorphic subclasses: `Weapon`, `Armor`, `Tool`, `Consumable`, `Misc`;
- Core domain entities: `Player`, `Listing` (market order), `Transaction` (completed trade record).

(2) Self-Implemented Data Structures Layer (`src/structures/`)

All six required data structures are implemented from scratch and deeply integrated with business logic:

- `HashMap` (separate chaining) → Repository ID index; - `Stack` (linked-list based) → CLI undo stack; - `Queue` (linked-list based) → FIFO batch settlement; - `DoublyLinkedList` → Player inventory backing store; - `CatalogTree` → Hierarchical item categories; - `PriceBST` → Sorted price range query on market.

(3) Service Layer (`src/services/`)

This layer implements business logic through well-defined service interfaces:

- `PlayerService`: manages player profile, gold balance, and CRUD; - `ItemService`: manages item catalog and category browsing; - `InventoryService` / `PlayerInventoryService`:

handles item addition, removal, stacking, equipping, and capacity enforcement; - **MarketService**: supports listing, delisting, searching, and purchasing items; - **TransactionService**: records an immutable trade history and produces analytics; - **Persistence**: handles JSON load-/save/seed/reset with integrity validation; - **logger**: provides unified structured logging.

(4) Exception Hierarchy Layer (`src/errors/`)

A three-level custom exception tree rooted at **TradingSystemError**, with three intermediate categories (**DataError**, **ValidationError**, **TradeError**) and 20 concrete exception classes. Service layer only raises; UI layer translates to user-friendly messages via a single **except TradingSystemError** handler.

(5) Persistence Layer (`data/`)

Stores state in five JSON files (`players.json`, `items.json`, `market.json`, `transactions.json`, `catalog.json`). On first run the system seeds a realistic dataset; subsequent runs load and update the existing data, with automatic backups before destructive operations.

(6) User Interaction Layer (`src/ui/`)

A command-line interface organised into domain-specific handlers (**PlayerHandler**, **ItemHandler**, **InventoryHandler**, **MarketHandler**, **ReportHandler**, **DataHandler**), driven by a central **TradingCLI** router. Validates input, calls service methods, catches typed exceptions, and formats output for the console.

- **How the User Interacts with the System**

The user performs an action through the CLI menu; the handler calls the corresponding service method; the service validates business rules and modifies domain objects; changes are persisted to JSON; the system returns success or an error message. A typical purchase flow: User → buy item → **MarketService** validates listing and funds → transfers item, updates gold, marks listing sold, creates **Transaction** → persists → shows result.

- **Why This Is a Coherent System, Not a Script**

The system is not a linear script because (1) concerns are strictly separated into six layers (models, structures, services, errors, persistence, UI) with single-responsibility modules; (2) abstract classes, mixins, and polymorphic dispatch enable type-safe extension without modifying existing code; (3) every service exposes fixed, documented method signatures with explicitly labelled side-effects and typed exceptions; (4) six self-implemented data structures are composed inside domain and service classes to match real access patterns. These qualities — modularity, extensibility, statefulness — distinguish a designed system from a sequence of commands.

2 Technical Design

2.1 Object-Oriented Modelling

- **Inheritance and Abstraction:**

The system implements strong abstraction and inheritance through a formal class hierarchy defined in the data model.

The core **abstract** class is `Item`, which acts as a base class for all in-game items and defines common properties, ranging from `item_id`, `name`, `category`, `rarity`, `base_value`, `to_dict()`, `from_dict()` to `describe()`.

Five concrete subclasses **inherit** from `Item`:

- `Weapon`
- `Armor`
- `Tool`
- `Consumable`
- `Misc`

This inheritance structure centralises shared behaviour in the base class while allowing specialised attributes in subclasses. Abstraction also appears in service interfaces, where high-level methods hide internal implementation details such as data structure operations and persistence logic.

In parallel, the other domain entities — `Player`, `Listing`, and `Transaction` — are concrete classes with well-defined boundaries: each owns its own state, exposes only service-level mutation methods, and serialises cleanly to and from JSON via `to_dict` / `from_dict`. Together with `Item`, they form the complete domain model.

- **Composition:**

Composition defines has-a relationships between entities, creating a cohesive and inter-connected object graph.

Key compositional relationships include:

- A `Player` has an `Inventory`
- An `Inventory` contains multiple `Item` instances
- A `Market Listing` belongs to a `Player` and references an `Item`
- A `Transaction` records a buyer, seller, and traded item

The inventory is implemented as a doubly linked list, and market listings use a binary search tree (BST) for sorted access. These data structures are composed inside service and domain classes, demonstrating how complex systems are built from independent, reusable objects.

- **Encapsulation & Polymorphism:**

Encapsulation

Encapsulation is achieved by bundling data and behaviour within classes while restricting direct external access to internal state.

All core entities — including `Player`, `Inventory`, `Item`, and `Listing` — store private fields such as `gold`, `durability`, `stack_size`, and internal node references. These values are not modified directly from outside the class. Instead, modification occurs only through well-defined service methods such as:

- `add_item()`
- `remove_item()`
- `spend_gold()`
- `update_listing_status()`

This protects data integrity, enforces business rules, and prevents invalid state changes. The service layer further encapsulates complex workflows such as trading and inventory management.

Polymorphism

Polymorphism enables uniform treatment of different item types while preserving type-specific behaviour.

Since `Weapon`, `Armor`, `Tool`, `Consumable`, and `Misc` all inherit from the abstract `Item` class, the inventory, market, and transaction systems process them interchangeably through the shared interface (`to_dict`, `from_dict`, `describe`). The most consequential use of polymorphism is deserialisation: `Item.from_dict` inspects the `category` field of a JSON record and dispatches to the appropriate subclass's constructor. As a result, `persistence.py` contains no `if category == "weapon"` branching — adding a new item type requires changes to `item.py` alone.

Behavioural mixins (`Durable`, `Equippable`, `Stackable`) add a second axis of polymorphism: weapons and tools share wear-and-tear semantics via `Durable`; consumables and misc items share stacking semantics via `Stackable`. This avoids the classical "diamond" problem of deep single-inheritance while still letting the inventory code ask `isinstance(item, Stackable)` to decide whether to merge slots.

- **Custom Exception Hierarchy:**

Following the course requirement for custom exceptions, we defined a three-level tree rooted at `TradingSystemError` (every expected failure inherits from it), with three intermediate categories — `DataError` (persistence / serialisation), `ValidationError` (input / business rule), and `TradeError` (trade-specific) — and 20 concrete exception classes below them. Each exception carries two channels: a Chinese `message` for end users, and a structured `context` dict (`player_id`, `listing_id`, `ref_id`, etc.) for logs and tests. The service layer only raises; the UI layer translates. This lets a single `except TradingSystemError` at the CLI boundary handle every expected error path uniformly, while any unexpected `Exception` is caught by an outer safety net so the program never crashes on a user action.

2.2 Data Structures and Algorithms

The six self-implemented structures were chosen by matching algorithmic complexity to expected workload, not picked at random from the required list:

2.3 Algorithms and Recursion

Two algorithmic patterns are worth calling out explicitly:

Recursive catalog traversal. `CatalogTree` represents items as a two-level tree (`root` → `category` → `subcategory`). Browsing a category recursively visits its subtree, yielding items in a depth-first order that matches how users mentally navigate menus. The recursion terminates at leaf nodes and carries no auxiliary data structure beyond the implicit call stack, keeping the implementation short and readable.

Range search on the price BST. `PriceBST.range_query(lo, hi)` prunes whole subtrees whose key ranges cannot possibly overlap `[lo, hi]`, yielding $O(\log n + k)$ time for k results — sharper than $O(n)$ linear filtering on the listings list. This is the hottest path on the market screen and the one place where the self-implemented structure most clearly outperforms a built-in alternative.

Polymorphic deserialisation dispatch. `Item.from_dict` inspects a record's `category` field, looks up the corresponding subclass, and delegates construction to it. This pattern turns

Structure		Complexity	Why This Structure
Doubly Linked List		$O(1)$ insert/delete	Inventory slot mutation without array shifting; preserves pickup order; enforces 50-slot capacity. A Python <code>list</code> would give $O(n)$ for removal at arbitrary positions.
Stack		$O(1)$ push/pop	CLI undo history stores lightweight snapshots for responsive rollback without full data duplication.
Queue		$O(1)$ enqueue/dequeue	FIFO batch settlement; one failed trade does not block the queue, maintaining system stability.
Tree		$O(n)$ traversal	Two-level category hierarchy (<code>weapon.sword</code>); recursive <code>walk</code> yields items depth-first in three lines, matching user mental model.
Binary Search Tree		$O(\log n)$ range	Market price filter: <code>range_query(lo, hi)</code> prunes subtrees, yielding $O(\log n + k)$ for k results vs $O(n)$ linear scan. The hottest path on the market screen.
Hash Map		$O(1)$ lookup	Repository indexes by <code>player_id</code> / <code>item_id</code> / <code>listing_id</code> ; every service pays this cost once, setting the floor on system responsiveness.

what could have been a tall `if / elif` chain into a single dictionary lookup, and it automatically absorbs any future item types added to the model layer.

3 Development and Testing

3.1 Iterative Development

The system was built incrementally over eight weeks using **GitHub** with a `main / dev / feat/*` branching model and pull-request review.

Iterations 1-2 — Contracts and Core Models. We produced three design contracts (`data-design.md`, `services-interface.md`, `error-and-log-design.md`) before code, fixing entity fields, method signatures, and the exception tree. A `Repository` container let services share five in-memory stores without passing six arguments. Team members implemented in parallel against stubbed interfaces. We then built the `Item` abstract class with three mixins and five subclasses, plus the six self-implemented data structures, each paired with its consuming service. Tests were written alongside code.

Iteration 3 — Persistence and Integration. A dedicated `Persistence` service replaced in-memory prototypes, adding `load_all` / `save_all` / `seed_if_empty` / `reset` and cross-entity integrity validation. `Item.from_dict` became a polymorphic factory instantiating the correct subclass from the `category` field — a deliberate OOP showcase in the deserialisation path.

Iterations 4-5 — UI and Stabilisation. The `TradingCLI` router and six handlers were built on the stable service layer. We enforced: services raise typed exceptions; only the UI translates them to Chinese. The final phase uncovered two blocking bugs (see §3.2), refactored the CLI into focused handlers, and back-filled UI tests from 0 to 133 cases, raising total coverage from 350 to 483 tests. All 61 feature-list items passed acceptance.

3.2 Problem Solving

Two significant defects surfaced during integration testing. Both were caught because the test suite reached across layers rather than stopping at unit boundaries — a direct payoff of the layered test strategy described in §3.3.

- **Problem 1 — Infinite loop on malformed stack limit.**

When an `Item`'s `stack_size_max` was loaded as 0 or a negative number (a realistic risk with hand-edited JSON or a buggy seed generator), `Inventory.add()` entered a non-terminating loop: the helper that splits an incoming quantity across slots never managed to consume any units because every attempted chunk was ≤ 0 , so the outer `while remaining > 0` condition never advanced. The program appeared to freeze with no error, which is far worse than crashing — the user has nothing to react to.

Diagnosis. A new boundary-case test in `test_inventory.py` hung rather than failing, which immediately localised the issue. Adding a `print` of `remaining` inside the loop confirmed it was stuck at a constant value. Reviewing the split logic showed the invariant "each iteration reduces `remaining`" was silently violated whenever `stack_size_max` ≤ 0 .

Fix. Rather than adding a loop-iteration cap (which would paper over the root cause), we added a pre-condition check at the top of `Inventory.add()`: if `stack_size_max` is non-positive, clamp it to 1. This restores the loop invariant and makes the behaviour predictable. A regression test with pytest's `timeout` marker now guards the boundary cases.

- **Problem 2 — Instance state lost during transfer.**

When an item moved between players (via market purchase or admin transfer), the `PlayerInventoryService.transfer_item()` call read only `item_id` and `quantity` from the source slot and called `add_item` on the destination with those two fields. Per-instance attributes stored on the slot — current durability, enchantment level, custom modifiers — were silently dropped, meaning a sword at 10/100 durability arrived at the buyer at 100/100.

Diagnosis. A transaction round-trip test compared pre- and post-trade item snapshots and flagged the mismatch. The `remove_item` path was also suspect: it filtered only by `item_id` and removed the earliest matching slot, so even a state-preserving transfer could pick the wrong slot.

Fix. `transfer_item` now reads the full slot payload (including `instance_state`) before removal and forwards it to `add_item` on the destination. `remove_item` accepts an optional slot identifier so callers that care about a specific physical slot can request it by reference. New tests verify that durability and modifiers survive a full `list` $\rightarrow$ `buy` $\rightarrow$ `deliver` flow.

Both fixes illustrate a shared lesson: when a bug survives unit tests, the problem is usually at a *boundary* — between data formats, between layers, or between services. Integration tests catch what unit tests cannot by construction.

3.3 Testing Strategy

The test suite contains **483 passing tests** across 23 files, distributed roughly as: data structures (76), domain models (33), services (229), CLI / UI (133), and application bootstrap (12). Testing was written alongside code, not appended afterwards, and it directly influenced several design decisions — for example, the dependency-injection hooks on `App` exist so `KeyboardInterrupt` and `DataIntegrityError` paths can be exercised without simulating keyboard input.

- **Unit Testing.** Isolated tests verify individual methods and classes. Test classes are grouped by *behavioural scenario* rather than by function name — for example, `test_persistence.py` organises cases into `TestSeedIfEmpty`, `TestLoadAll`, `TestRoundTrip`, `TestIdCounters`, `TestIntegrityCheck`, `TestErrorPaths`, and `TestBackupAndReset`. This makes it easy to demonstrate coverage of a concern ("seven persistence scenarios") rather than of a

surface area ("nineteen functions"). All file-system tests use `pytest's tmp_path` fixture so they never touch the real `data/` directory.

- **Module / Integration Testing.** These tests verify that components cooperate correctly: that `MarketService.buy` atomically updates four entities (buyer gold, seller gold, inventory contents, listing status), that `Persistence.save` → `load` is a true round-trip, and that every `Transaction` references live `Player` and `Item` IDs. Exception assertions check the `type` and the structured `context` dict rather than the user-facing message string, so UI copy changes do not cascade into test failures.
- **System Testing.** End-to-end tests drive the CLI through scripted input, covering the golden purchase flow (create player → deposit gold → list item → buy → verify transaction record → persist → reload → verify state), the data-management sub-menu (immediate save, backup, reset), and error recovery (invalid input, missing records, insufficient funds). These tests catch cross-layer regressions that unit tests cannot see.
- **Debugging-Driven Design.** Several design refinements came directly from test failures: the `stack_size_max` clamp, the slot-level `transfer_item` signature, the display cap on long report outputs (top 20 shown, "N more hidden"), and the separation of `Inventory` (structural operations on `DoublyLinkedList`) from `PlayerInventoryService` (business-rule orchestration) all emerged from tests that were painful to write against the original design. In each case, we changed the production code rather than the test — a sign the tests were exercising real weaknesses.

4 Reflection

4.1 Critical Assessment

- **What the system does well**

Clear entity relationships and a stable data contract. The five JSON files correspond one-to-one with the five core entities (`Player`, `Item`, `Listing`, `Transaction`, `Catalog`), with explicit ID prefixes (`p_`, `i_`, `l_`, `t_`) and a monotonically-increasing counter per type. Cross-entity references are validated on load, so an incomplete or tampered data file raises `DataIntegrityError` before any business code runs on it. This contract was fixed early via `data-design.md` and survived every subsequent iteration unchanged.

Data structures chosen to match access patterns, not for demonstration. Each of the six self-implemented structures earns its place: `HashMap` for $O(1)$ ID lookup in the repository, `PriceBST` for $O(\log n)$ range queries on market listings, `DoublyLinkedList` for $O(1)$ slot-level mutation in the inventory, `CatalogTree` for recursive category browsing, `Stack` for the undo history, `Queue` for FIFO batch settlement. Rewriting the system on top of built-in `list` / `dict` would degrade both the price range query and the undo feature noticeably.

Polymorphic deserialisation as a real OOP payoff. `Item.from_dict` inspects the `category` field and dispatches to the correct subclass's constructor. `persistence.py` therefore contains no `if category == "weapon"` branches — the polymorphism is real, not decorative, and adding a sixth item type would touch only `item.py`.

Disciplined separation between raising and translating exceptions. The service layer only raises typed exceptions; the UI layer is the sole place that translates them into Chinese user-facing messages. This pays off twice: service-layer tests assert

on structured `context` dicts so UI copy tweaks do not cascade, and a single `except TradingSystemError` at the menu boundary handles every expected error uniformly.

Tests that drove design, not just verified it. Several architectural choices — dependency-injected `ui_runner`, slot-level `transfer_item`, the `stack_size_max` clamp — exist because tests made them necessary. The 483 tests are not a post-hoc safety net; they shaped what the production code looks like.

- **Current limitations**

Documentation-implementation drift during the middle iterations. We wrote `services-interface.md` before most services existed, which was useful for parallelisation but created a lag: several method signatures diverged from the document, and the functions list briefly conflated "entry-point exists" with "fully implemented". A late-stage code-review pass resolved the drift, but a more disciplined team would keep docs and code in sync continuously.

Flat-file persistence is a ceiling, not a floor. JSON was the right choice for this project — it kept the focus on self-implemented data structures rather than deferring to an SQL engine — but it will not scale beyond a few thousand records per file. Concurrent access, partial updates, and transactional multi-file writes all require workarounds. A real deployment would need at least SQLite.

CLI-only interaction constrains what we can demonstrate. The terminal is fine for correctness demos but awkward for exploring large catalogs or multi-step trades. Pagination and display caps mitigate the problem; they do not solve it.

No CI, no static type checking, no coverage gate. Tests run locally before commit. There is no automated check on merge, no `mypy` run, and no coverage report. For a course project of this scope it is acceptable; for a team that plans to keep iterating it is the first thing we would add.

4.2 Future Improvements

Our plans for extending the system are architectural, not just feature-additions.

Swap flat JSON for SQLite behind the existing Persistence interface. Because all storage access is already funnelled through one class with a stable signature, moving to SQLite is a localised change: the service layer stays untouched. This would remove the file-locking and concurrent-write risks and let us query with real SQL where appropriate (e.g., complex report joins), while keeping the self-implemented data structures alive for their original roles as in-memory caches and undo history.

Split Inventory cleanly from PlayerInventoryService. The current arrangement works but leaks responsibilities: `Inventory` owns the `DoublyLinkedList`, but some slot-shape decisions (stacking rules, instance-state preservation) live in `PlayerInventoryService`. A cleaner split would make `Inventory` a pure structural adapter and promote all business rules into the service. This is the kind of refactor that the test suite would make safe.

Front-end and HTTP layer. A thin Flask / FastAPI layer exposing the service interface would enable a browser UI without rewriting any business logic — the service layer was designed without CLI-specific assumptions precisely so this migration is possible. This also opens up real multi-player scenarios that a single-user CLI fundamentally cannot support.

Continuous integration with coverage reporting. GitHub Actions running `pytest`, `mypy`, and a coverage gate on every pull request. This would catch documentation-drift and regression risks earlier and make the 483-test suite useful as a merge gate rather than a manual checklist.

Self-balancing BST. The current `PriceBST` is unbalanced; worst-case degeneration to $O(n)$ is a latent risk if listings arrive in sorted order. Upgrading to a self-balancing variant (AVL or red-black) would guarantee $O(\log n)$ without changing the service-layer API.

4.3 Learning Outcomes

The clearest lesson for us was that **OOP is a design tool, not a style guide**. It is easy to produce code that looks object-oriented — classes everywhere, inheritance trees — and gets none of the real benefits. The parts of our system where OOP actually paid off (polymorphic deserialisation, the layered architecture enabling parallel development, the exception hierarchy letting one `except` clause cover many error types) all came from asking "what problem is this solving?" rather than "can I use a class here?". Conversely, we were repeatedly tempted to build premature abstractions (a `BaseService`, a generic `Repository<T>`) and repeatedly glad we resisted; by the time a need was real, we understood it well enough to build the right thing instead of the imagined thing.

We also learned, concretely, that **data structure choice is a design decision with observable consequences**. Writing our own `HashMap` and `PriceBST` forced us to think about what each structure actually provides — not just its Big-O numbers but the invariants it maintains. When we picked the wrong one (an early version used a list scan for price range queries), the test suite got visibly slower. Replacing it with the BST was an immediate, measurable improvement. We now read textbook complexity tables with a different eye: not as trivia, but as a prediction about the system we are about to build.

On the process side, the project taught us how much leverage a **design contract** provides in a team. Writing `data-design.md` / `services-interface.md` / `error-and-log-design.md` before the code felt slow at the time and was indispensable in hindsight: six people could implement in parallel because every boundary was already negotiated. When contracts drifted from code, we felt it. The lesson is not "always write docs first" — it is "when collaboration is the bottleneck, contracts are the cheapest synchronisation primitive you have".

Finally, we learned that **tests are a design tool, not just a correctness tool**. The tests that gave us the most value were not the ones verifying a single method in isolation but the ones that refused to fit until we restructured production code — the `KeyboardInterrupt` path that forced dependency injection, the transfer-state round-trip that forced a slot-level API. The test we could not write was the bug we could not fix.